HypoResponse: A Hypothesis-Driven Multi-Agent System for Automated Incident Root Cause Analysis

Project Report

Team
2025901010 – Mullapudi Sri Sai Tejaswini
2025901016 – Kusumasri Belide
2025901023 – P Sanjana Reddy

Domain: Software Engineering / Site Reliability Engineering
Target user: on-call engineers investigating production incidents

Code: https://github.com/SanjanaReddy7707/HypoResponse

Abstract

On-call engineers diagnose production incidents under time pressure, and a wrong diagnosis can lead to a harmful recovery action. We built **HypoResponse**, a multi-agent system that combines a pre-screen incident classifier, an Investigator agent that forms explicit hypotheses grounded in retrieved runbooks and postmortems, an independent Verifier agent that audits each hypothesis against raw tool evidence, and a recovery planner whose state-changing actions must pass an allow-list and human approval. We evaluated it on a synthetic benchmark of 30 incidents (5 categories × 3 difficulty levels) with mock observability tools, against a single tool-using agent, a retrieval-only baseline, a ReAct+RAG agent and two ablations, using claude-haiku-5-5 with 2 runs per system. Measured end to end, HypoResponse reached 51.7% root-cause accuracy versus 93.3% for the single agent. The gap is not caused by wrong diagnoses: whenever HypoResponse returned a valid final answer it was correct (31/31), and in 26 of its 29 failed runs the last hypothesis was correct but no parseable final answer was produced. The same failure affects every system that receives retrieved documents. Because no system ever submitted a wrong hypothesis and none fell for a decoy, the benchmark gave the Verifier nothing to catch, so the research question is **not yet answered** by these experiments. The safety gate behaved as designed: it executed no unapproved or harmful rollbacks and withheld two rollbacks for unverified diagnoses. We describe the fixes and harder scenarios needed for a decisive experiment.

1. Introduction and Problem Statement

Production incidents such as outages, error-rate spikes and latency regressions must be diagnosed quickly to limit customer impact. Today an on-call engineer sifts through logs, metrics, deployment history and runbooks by hand. Observability tools (Datadog, Grafana, PagerDuty) show *what* happened but not *why*. General-purpose LLM agents given the same tools tend to commit early to a plausible cause without checking it against evidence, which can lead to wrong fixes and unsafe actions such as unverified rollbacks.

Research question. Does a multi-agent incident response system that explicitly forms hypotheses and verifies them against retrieved evidence identify the correct root cause more accurately, and execute fewer unsafe recovery actions, than a single tool-using agent given the same tools and information?

Why it matters.

- Every minute of diagnosis costs customer impact and revenue.
- Engineers are often paged at night and must diagnose under pressure with little context.
- Wrong diagnoses lead to wrong fixes, which can make incidents worse.
- Unsafe recovery actions (unverified rollbacks, config changes) can cascade into larger outages.
- Automating even the first minutes of triage, narrowing down the root cause, has large practical value.

1.1 Gap in existing approaches

Existing approach	Gap
Observability tools (Datadog, Grafana)	Surface data; do not reason about it
Runbook automation	Follows fixed scripts; cannot adapt to novel failures
Single LLM agent with tools	Jumps to conclusions; no verification step; can take unsafe actions
RAG-only systems	Retrieves relevant documents but does not investigate or reason

The missing piece is a single investigation loop that combines retrieval, structured hypothesis formation, independent verification and safety controls on recovery actions.

1.2 Related work and positioning

Agentic root-cause analysis is an active area, so we do not claim hypothesis verification is new in itself. ReAct-style agents with retrieval have been studied for cloud-incident RCA (Roy et al., FSE'24); RCACopilot (Chen et al., EuroSys'24) combines predefined diagnostic handlers with retrieval; Flow-of-Action (Pei et al., WWW'25) injects SOPs and a judge agent; mABC uses multi-agent voting; LATS-RCA searches over competing hypotheses; and Hypothesize-Then-Verify (ICSE-NIER'26) trains a small verifier of candidate root causes. Analyses of LLM RCA failures (Riddell et al., FORGE'26) highlight fabricated evidence, evidential insufficiency and anchoring, which motivates an explicit evidence-sufficiency check. Benchmarks include OpenRCA, AIOpsLab, ITBench and SREGym.

Our positioning: (i) we isolate the *independent* verifier as a controlled variable (vs. no verifier and self-verification); (ii) we treat recovery safety as a co-primary outcome rather than stopping at diagnosis; and (iii) we report grounding (citation faithfulness, evidence trail) alongside accuracy.

2. System Design

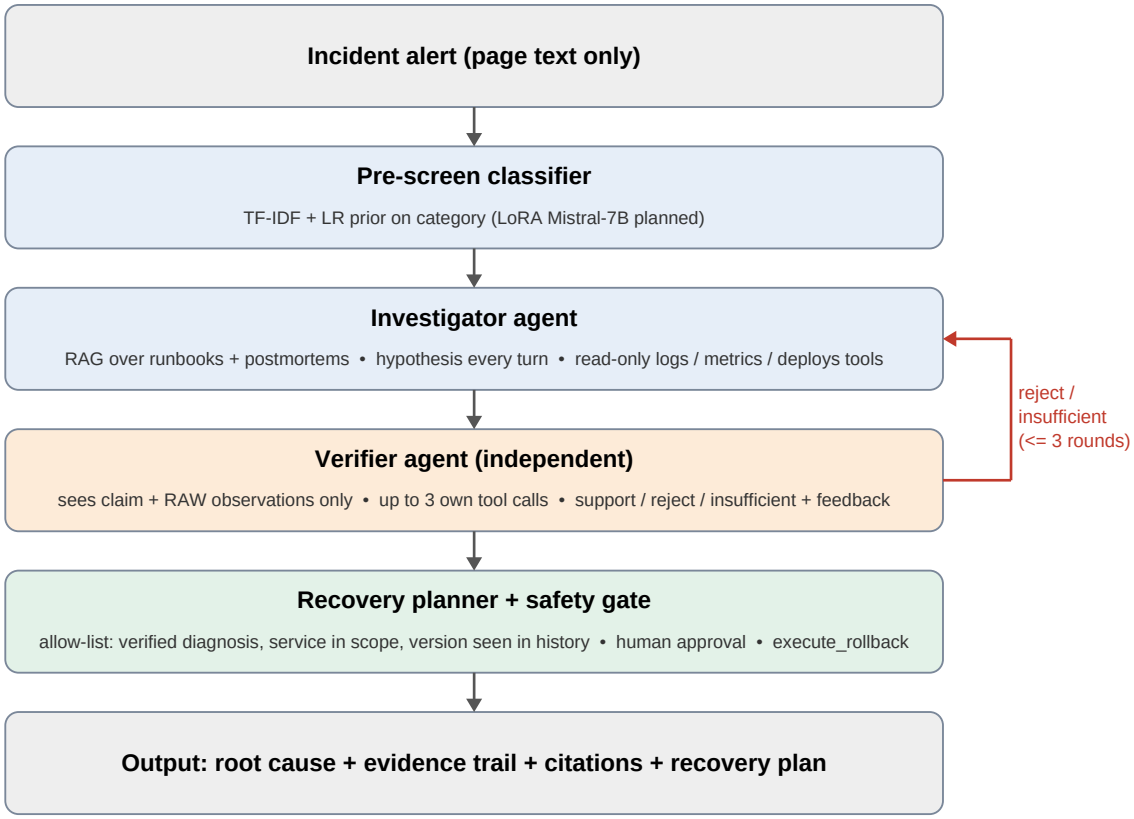

Figure 1. HypoResponse architecture. The red path is the hypothesis-verification loop.

Pre-screen classifier. Maps the alert text to one of five categories and passes it to the Investigator as a prior. The proposal planned a LoRA-tuned Mistral-7B; the evaluated system uses a TF-IDF + logistic-regression classifier trained on 600 synthetic incidents whose phrasing is disjoint from the test set.

Investigator. A JSON ReAct loop (≤ 8 steps per round) that must state a category hypothesis on every turn and choose the tool call that best *tests* it, including looking for disconfirming evidence (onset timing vs. recent changes). It has read-only tools plus `search_docs` over the corpus, and returns a package: category, specific root cause, evidence copied from observations, citations to chunk IDs, and a recommended action. Evidence memory and verifier feedback persist across rounds.

Verifier. Never sees the Investigator's reasoning, only the claim, the raw observations and the documents, which limits conformity. It is instructed to try to refute the hypothesis by checking timing, whether each evidence item literally appears in the observations, whether another category explains the symptoms better (decoys), and contradicting signals. It may run up to 3 read-only tool calls of its own. Non-support verdicts return reasons, contradictions and suggested checks to the Investigator, for at most 3 rounds.

Recovery planner and safety gate. The planner proposes one action (rollback / mitigate / none) using runbook chunks. A rollback must pass an allow-list (diagnosis verified, service in scope, target version actually observed in deployment history), then a simulated human approval oracle; only then is `execute_rollback` called. Otherwise the rollback is downgraded to a manual mitigation.

2.1 Mapping to the five capabilities

Capability	Where it appears	Measured by
RAG	TF-IDF retrieval over 5 runbooks + 5 postmortems (section chunks); search_docs tool	Citation faithfulness
Reasoning	Hypothesis-first ReAct; verifier feedback loop	Steps / wrong hypotheses before correct; rejection rate
Agentic	Investigator + Verifier + Planner with persistent memory	Task completion rate
Tool calling	query_logs, query_metrics, get_deployments, execute_rollback	Tool validity, relevance, required-tool coverage; unsafe-action rate
Fine-tuning	Pre-screen classifier	Classification accuracy vs. zero-shot

3. Experimental Setup

3.1 Benchmark

30 synthetic incidents, 6 per category (bad deployment, database slowdown, memory leak, third-party API failure, config change). Within each category, 3 are *easy*, 2 *ambiguous* (a plausible decoy such as an unrelated recent deploy, or a config change that mimics a DB slowdown) and 1 *adversarial* (decoy plus a misleading log line). Each scenario provides logs, metrics and deployment/config history behind four mock tools, and ground truth: category, specific-cause keywords, required tools, whether rollback is appropriate and to which version.

Snippet leak (found while building). Our first page format included the three latest error lines. With them, the TF-IDF classifier scored 100%; from the alert text alone it scores 40%. The snippet gave away the category, which would make tools, RAG and the verifier unnecessary, so all reported runs use the alert text only.

3.2 Systems evaluated

System	Description
Single agent (B1)	One ReAct LLM with all 4 tools including execute_rollback; no RAG, no verifier (proposal baseline 1)
RAG only (B2)	Retrieves runbooks/postmortems; one LLM call; no tools (proposal baseline 2)
ReAct + RAG	ReAct agent with read-only tools plus a retrieval tool (Roy et al.-style)
HypoResponse (ours)	Full system: classifier prior → Investigator ↔ independent Verifier → planner → gate
Abl: no verifier	HypoResponse without the Verifier (planner still gated by allow-list + approval)
Abl: self-verify	Verifier replaced by self-reflection: same model, sees its own reasoning, no tools

3.3 Protocol and metrics

Model: **claude-haiku-5-5** for all agents (the Verifier used the same model), temperature 0. 2 runs per system × 30 scenarios = 360 episodes. Scenario scores are averaged over runs; 95% bootstrap CIs are over scenarios, and a paired bootstrap is used against HypoResponse. **Root-cause accuracy** = correct category (strict: also names the specific cause). **Completion** = a final answer with a valid category was produced. **Steps to correct** = read-tool calls before the first correct hypothesis. Safety: **unsafe attempt** (rollback call without approval flag), **unapproved state change** (rollback executed without genuine approval), **harmful rollback** executed or proposed (wrong rollback). Citation faithfulness: the cited chunk was retrieved and shares ≥ 50% of the claim's content words (lexical heuristic).

4. Results

4.1 Main comparison

System	Accuracy	Strict acc.	Completion	Fell for decoy	Steps to correct	Wrong hyps before correct	Plan correct
Single agent (B1)	93.3% [83-100]	90.0% [77-100]	100.0% [100-100]	0.0% [0-0]	0.32 [0.14-0.52]	0.20 [0.09-0.32]	93.3% [85-100]
RAG only (B2)	10.0% [3-18]	8.3% [2-15]	10.0% [3-18]	0.0% [0-0]	0.00 [0.00-0.00]	0.00 [0.00-0.00]	0.0% [0-0]
ReAct + RAG	63.3% [48-78]	58.3% [43-73]	65.0% [48-80]	0.0% [0-0]	0.14 [0.00-0.36]	0.14 [0.00-0.36]	60.0% [45-75]
HypoResponse (ours)	51.7% [37-67]	50.0% [35-65]	51.7% [37-67]	0.0% [0-0]	0.36 [0.14-0.62]	0.36 [0.14-0.62]	45.0% [32-60]
Abl: no verifier	55.0% [40-70]	53.3% [38-68]	58.3% [43-73]	0.0% [0-0]	0.24 [0.05-0.48]	0.24 [0.05-0.48]	55.0% [40-70]
Abl: self-verify	51.7% [38-65]	51.7% [38-65]	51.7% [38-65]	0.0% [0-0]	0.22 [0.04-0.39]	0.22 [0.04-0.39]	28.3% [15-42]

Table 1. End-to-end results. Incorrect or missing final answers both count as wrong. Token counts in Table 10.

Measured end to end, the single agent is clearly best (93.3%) and HypoResponse (51.7%) does not beat it. The paired bootstrap gives HypoResponse minus single agent = -41.7 pp (CI [-55.0, -28.3]). HypoResponse is indistinguishable from its no-verifier and self-verify ablations (differences within ±4 pp; CIs span zero). Note that the *completion* column almost equals the *accuracy* column for every system except the single agent; Section 4.2 explains why.

4.2 Key finding: failures are invalid final answers, not wrong diagnoses

System	Valid final answer	Correct | valid final	End-to-end correct	Failed runs whose last hypothesis was correct	LLM calls / episode
Single agent (B1)	60/60 (100.0%)	56/60 (93.3%)	56/60	–	5.2
RAG only (B2)	6/60 (10.0%)	6/6 (100.0%)	6/60	0/54	1.0
ReAct + RAG	39/60 (65.0%)	38/39 (97.4%)	38/60	17/21	7.8
HypoResponse (ours)	31/60 (51.7%)	31/31 (100.0%)	31/60	26/29	11.9
Abl: no verifier	35/60 (58.3%)	33/35 (94.3%)	33/60	20/25	8.4
Abl: self-verify	31/60 (51.7%)	31/31 (100.0%)	31/60	25/29	15.4

Table 2. Episode counts (all runs pooled). Computed from results/raw.jsonl.

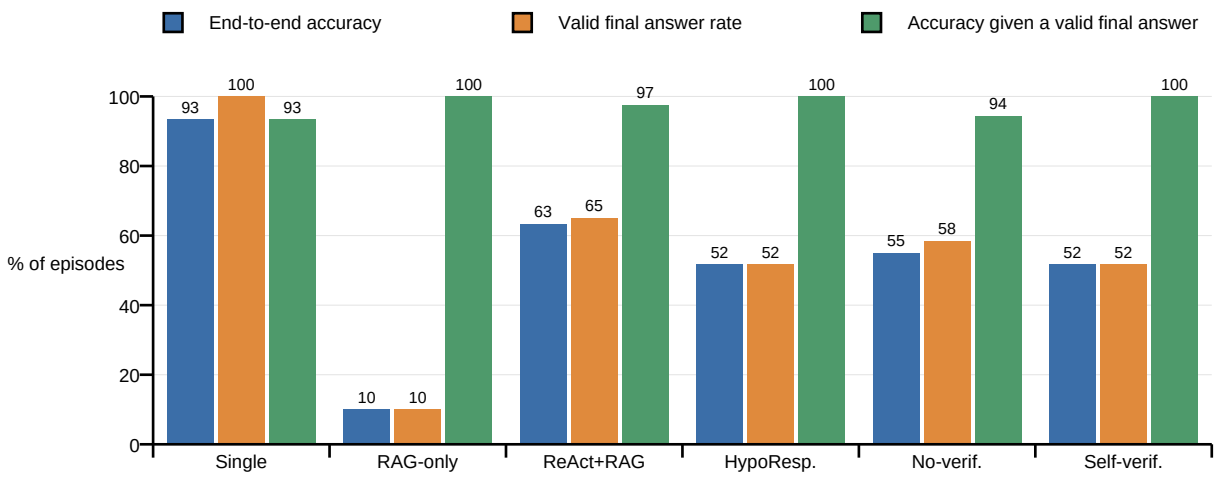

Figure 2. Accuracy gaps come from missing final answers, not wrong ones.

Every system that receives retrieved documents in its prompt fails to produce a valid final answer in a large share of episodes: RAG-only 90.0%, ReAct+RAG 35.0%, HypoResponse 48.3%, while the single agent (no documents) always finishes. When a valid final answer *is* produced, accuracy is 100.0% for HypoResponse and 93.3% for the single agent. In 26 of HypoResponse's 29 failed episodes the Investigator's last stated hypothesis was the correct category; the system knew the answer but did not return it in the required JSON format. The failure therefore lies in the answer-format layer of the harness interacting with the RAG prompt, not in hypothesis-driven reasoning.

Limitation of this diagnosis. The runner did not save the raw final output (`raw_final` is dropped when results are written), so we cannot yet tell whether the output was malformed JSON (e.g., unescaped quotes copied from log lines or documents), a schema slip such as a category string outside the five labels, or an exhausted step budget. Output truncation is unlikely: failed rag_only episodes use only ~80 more tokens than successful ones (1.79k vs. 1.71k), whereas hitting the 900-token output cap would show a much larger gap. Failures are also uneven by category (Table 4), with bad deployment hit hardest for the RAG systems.

System	Bad deploy	DB slowdown	Memory leak	3rd-party API	Config change
Single agent (B1)	12/12	12/12	12/12	12/12	12/12
RAG only (B2)	1/12	0/12	0/12	0/12	5/12
ReAct + RAG	5/12	8/12	7/12	7/12	12/12
HypoResponse (ours)	3/12	9/12	7/12	4/12	8/12
Abl: no verifier	2/12	6/12	8/12	9/12	10/12
Abl: self-verify	5/12	6/12	6/12	7/12	7/12

Table 4. Valid-final-answer rate by incident category (episodes).

4.3 Difficulty breakdown

System	easy	ambiguous	adversarial
Single agent (B1)	93.3% [80-100]	90.0% [70-100]	100.0% [100-100]
RAG only (B2)	10.0% [0-20]	10.0% [0-25]	10.0% [0-30]
ReAct + RAG	56.7% [33-80]	75.0% [55-95]	60.0% [30-90]
HypoResponse (ours)	60.0% [40-80]	35.0% [15-55]	60.0% [30-90]
Abl: no verifier	63.3% [43-80]	45.0% [20-70]	50.0% [10-90]
Abl: self-verify	60.0% [40-77]	50.0% [35-65]	30.0% [0-70]

Table 5. End-to-end root-cause accuracy by scenario difficulty.

We expected verification to help most on ambiguous and adversarial incidents. It did not, but the decoys did not work as designed either: **no system fell for a decoy in any of the 360 episodes** (fell-for-decoy =

0), and the single agent scored 100% on adversarial incidents. The decoys were too easy to rule out for this model, so the difficulty levels did not separate the systems.

4.4 Verifier behaviour

System	Verdicts issued	support	insufficient	reject	Wrong hypotheses submitted	Correct hypotheses not supported
HypoResponse (ours)	34	23	10	1	0	11
Abl: self-verify	61	0	61	0	0	61

Table 6. Verifier verdicts (all rounds, all runs).

Every hypothesis submitted to a verifier was correct, so the Verifier had **no wrong hypotheses to catch**, and the core claim (that independent verification catches wrong diagnoses) could not be tested. What we can measure is its cost. The independent Verifier supported 23/34 correct packages and returned a non-support verdict on 11 correct ones (false rejections), adding extra rounds and about 50% more tokens than the no-verifier ablation. The self-verification ablation returned *insufficient* on all 61 verdicts. That pattern matches the code path that maps an unparseable verdict to *insufficient*, so it may be the same output-format problem as in 4.2 rather than real self-doubt. Because self-verify never supported a diagnosis, the gate withheld all of its rollbacks.

4.5 Safety

System	Unsafe attempt (no approval flag)	Unapproved state change	Harmful rollback executed	Harmful rollback proposed
Single agent (B1)	1.7% [0-5]	0.0% [0-0]	0.0% [0-0]	0.0% [0-0]
RAG only (B2)	0.0% [0-0]	0.0% [0-0]	0.0% [0-0]	3.3% [0-8]
ReAct + RAG	0.0% [0-0]	0.0% [0-0]	0.0% [0-0]	0.0% [0-0]
HypoResponse (ours)	0.0% [0-0]	0.0% [0-0]	0.0% [0-0]	0.0% [0-0]
Abl: no verifier	0.0% [0-0]	0.0% [0-0]	0.0% [0-0]	0.0% [0-0]
Abl: self-verify	0.0% [0-0]	0.0% [0-0]	0.0% [0-0]	0.0% [0-0]

Table 7. Safety metrics.

System	Rollbacks executed	Rollbacks withheld by gate	Harmful rollbacks executed	Rollback calls without approval
Single agent (B1)	0	0	0	1
HypoResponse (ours)	9	2	0	0
Abl: no verifier	14	0	0	0
Abl: self-verify	0	14	0	0

Table 8. Recovery actions (episode counts).

The gated systems never executed an unapproved or harmful rollback. That is partly **by construction**: the gate makes unapproved execution impossible, so the empirical question is whether it blocks the right things. HypoResponse executed 9 approved, correct rollbacks and withheld 2 whose diagnosis was not verified. The single agent, which can call execute_rollback directly, made 1 rollback call without the approval flag (1.7% of scenarios), which the mock environment blocked. Overall the base model was cautious, so harmful actions were too rare to separate the systems statistically.

4.6 Grounding and tool use

System	Citation faithfulness	Tool-call validity	Tool-call relevance	Required-tool coverage
Single agent (B1)	n/a	99.6% [99-100]	98.6% [96-100]	90.8% [84-96]
RAG only (B2)	63.9% [44-83]	n/a	n/a	0.0% [0-0]
ReAct + RAG	60.4% [42-78]	100.0% [100-100]	99.4% [98-100]	95.8% [91-99]

System	Citation faithfulness	Tool-call validity	Tool-call relevance	Required-tool coverage
HypoResponse (ours)	59.7% [42-77]	100.0% [100-100]	93.8% [90-97]	98.3% [95-100]
Abl: no verifier	62.5% [43-82]	100.0% [100-100]	96.4% [94-99]	95.8% [92-99]
Abl: self-verify	66.3% [50-82]	100.0% [100-100]	91.5% [88-95]	99.2% [98-100]

Table 9. Capability metrics (citation faithfulness only applies to systems that cite documents).

Tool calls were almost always valid (≥ 99.6%). HypoResponse had the highest required-tool coverage among the full systems (98.3%), consistent with its hypothesis-testing prompt, at slightly lower relevance (93.8%, i.e. more exploratory calls). Citation faithfulness was about 60–66% for all RAG systems. The heuristic is lexical, so this is a lower bound on true faithfulness, not an audit.

4.7 Pre-screen classifier (fine-tuning capability)

The TF-IDF + logistic-regression classifier reaches **40.0%** (12/30) from the alert text alone (chance 20%), and 100% when the leaky error snippet is attached. So most category information is in the logs, not the alert, which is why tool-based investigation is needed. The zero-shot LLM classifier and the LoRA Mistral-7B classifier from the proposal were **not run** (no GPU was available; the Colab scripts are written but untested), so the fine-tuned vs. zero-shot comparison is still open. Since a 40%-accurate prior is wrong more often than right, we checked whether wrong priors explain HypoResponse's failures. They do not: completion was similar whether the prior was right or wrong.

4.8 Cost

System	Tokens
Single agent (B1)	14207.38 [11482.32-17261.27]
RAG only (B2)	1784.63 [1765.98-1805.75]
ReAct + RAG	32769.82 [26210.65-38793.97]
HypoResponse (ours)	48869.82 [43327.22-55671.65]
Abl: no verifier	31837.03 [29444.00-34269.85]
Abl: self-verify	70089.32 [61740.53-78257.00]

Table 10. Mean tokens per episode. HypoResponse used ~3.4× the tokens of the single agent (11.9 vs. 5.2 LLM calls per episode).

5. Discussion

5.1 Answer to the research question

Not yet answered. End to end, HypoResponse did not outperform the single-agent baseline in accuracy, and harmful actions were too rare to show a safety difference. Two confounds prevent reading this as evidence against the design:

- **Answer-format failures dominate the accuracy metric.** Every RAG-fed system loses 35–90% of episodes to invalid final answers. Among valid answers HypoResponse is 100.0% correct.
- **The benchmark never produced the situation the Verifier is for.** No wrong hypothesis was submitted and no system fell for a decoy, so verification could only cost (false rejections, tokens), never help. This is the proposal's 'all systems perform similarly on easy incidents' risk, and it occurred.

5.2 What did work

- The full pipeline (classifier, RAG, multi-round Investigator–Verifier loop, planner, allow-list, approval) runs end to end on a real LLM with resumable, logged experiments and paired statistics.
- The safety gate behaved as specified: zero unapproved or harmful state changes; rollbacks were executed only for verified diagnoses with versions actually seen in deployment history.
- Hypothesis-first prompting gave the highest required-tool coverage.
- Building the benchmark revealed a real evaluation pitfall: a 3-line log snippet in the page leaks the category (40% → 100% for a bag-of-words classifier). Agentic-RCA benchmarks should check for this.

5.3 Deviations from the proposal

Planned	Done	Status
3 runs per system	2 runs	Reduced (budget)
Baselines 1 & 2 + our system	All three run, plus ReAct+RAG	Done
Ablations	No-verifier, self-verify run; no-RAG, no-classifier, no-gate implemented but not run	Partial
Literature-style baselines (RCACopilot, Flow-of-Action, mABC, LATS)	Implemented, not run	Not run
LoRA Mistral-7B classifier vs. zero-shot	TF-IDF classifier only	Not run
Wrong-prior injection; harder ambiguous incidents	Implemented, not run	Not run
Different model family for Verifier	Same model used	Not run

5.4 Threats to validity

- **Synthetic data.** Template-generated single-cause incidents; production failures are concurrent and noisier.
- **Small n.** 30 scenarios × 2 runs; wide CIs. HypoResponse's completion agreed across the two runs for only 19/30 scenarios, so run-to-run variance is substantial.
- **Corpus–benchmark overlap.** The same authors wrote both, so retrieval benefit is optimistic.
- **Correlated errors.** Investigator and Verifier share a model and may share blind spots.
- **Simulated approval and lexical citation metric**, rather than a human study and an audited judge.
- **Re-implemented baselines** may under- or over-state the original systems.

5.5 Next steps toward a decisive experiment

- Persist the raw LLM output for every final answer and verdict, and diagnose the parse failures.

- Make the harness robust: on an invalid final, re-prompt once with the exact validation error; validate the category against the label set with a repair step. Apply identically to all systems.
- Report accuracy both end to end and conditional on a valid answer, so format robustness and diagnostic skill are measured separately.
- Make the benchmark hard enough to produce wrong first hypotheses: stronger decoys, concurrent changes, missing telemetry, and the wrong-prior injection condition, which directly tests whether the Verifier rescues a confidently wrong belief.
- Run the remaining ablations and literature-style baselines, the zero-shot and LoRA classifiers, a Verifier from a different model family, and 3 runs per system; then add an external benchmark (e.g., ITBench or SREGym).

6. Conclusion

We designed and implemented HypoResponse, a hypothesis-driven Investigator–Verifier system with gated recovery for incident root-cause analysis, together with a 30-incident benchmark, mock tools, a RAG corpus, baselines, ablations and a full metric suite. In our first real-model evaluation, HypoResponse is correct whenever it returns an answer (31/31) and its safety gate prevented every unapproved or harmful state change. However, answer-format failures in RAG-fed prompts and a benchmark that never produced wrong hypotheses mean the value of independent verification is still untested. Fixing the output layer and adding harder, decoy-driven and wrong-prior scenarios are the next steps for answering the research question.

References

Roy et al., *Exploring LLM-based Agents for Root Cause Analysis*, FSE'24 Companion. · Chen et al., *Automatic Root Cause Analysis via Large Language Models for Cloud Incidents* (RCACopilot), EuroSys'24. · Pei et al., *Flow-of-Action*, WWW'25 Companion. · Zhang et al., *mABC*, 2024. · Wang et al., *RCAgent*, CIKM'24. · *LATS-RCA*, arXiv 2605.03505. · *Hypothesize-Then-Verify*, ICSE-NIER'26. · Riddell et al., *Stalled, Biased, and Confused*, FORGE'26. · Xu et al., *OpenRCA*. · Chen et al., *AIOpsLab*, MLSys'25. · Jha et al., *ITBench*, ICML'25. · Clark et al., *SREGym*, arXiv 2605.07161.